



Applied Cryptography

Summer 2026

Part 1: Provable Security

1.4: Pseudorandomness

Nadim Kobeissi

<https://appliedcryptography.page>

Section 1

Pseudorandom Generators

Limitations of One-Time Pad

The Key Length Problem

One-time pad is not a particularly useful encryption scheme in practice.

- The key must be as long as the plaintext!
- This creates a chicken-and-egg situation:
 - To privately send n bits of information,
 - We must already privately share n bits of information.
- Impractical for most real-world applications.
 - Clearly this is not what we're doing when we use HTTPS,
 - or WhatsApp, or pay for something via a debit card...
- We need encryption schemes where the key can be smaller than the message.

Idea: find a way to expand the key

- Given a key k_s of size $|k_s| < |m|$, find a way to obtain $|k_e| \geq |m|$
- In the real world, we have two kinds of symmetric encryption schemes:
 - **Block ciphers:** AES, 3DES, etc.
 - **Stream ciphers:** ChaCha20, RC4, etc.
- This is exactly what stream ciphers do!
 - Start with a small key k_s of a fixed size $|k_s| = \lambda$,
 - Magically expand it to k_e where $|k_e| \geq |m|$,
 - $\text{ENC}(K, M) = K \oplus M$

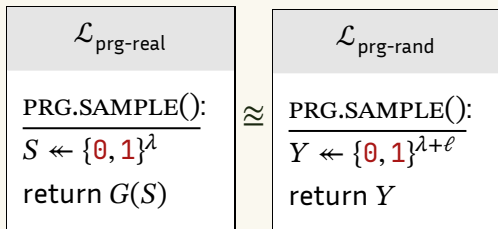
Requirements for Key Expansion

- Our method would need to be **deterministic**, so that both the sender and receiver can expand their key in the same way (to encrypt/decrypt).
- Its output distribution would need to be **uniform**, since that is a crucial property for the security of OTP.
- Unfortunately, it's **not possible** to achieve both of these properties simultaneously.
 - Suppose the expansion method is a deterministic function $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+\ell}$
 - Its outputs are ℓ bits longer than its inputs!
 - There are $2^{n+\ell}$ strings of length $n + \ell$ but only (at most) 2^n possible outputs of G
 - So the outputs of G can never induce a uniform distribution.

Enter pseudorandomness

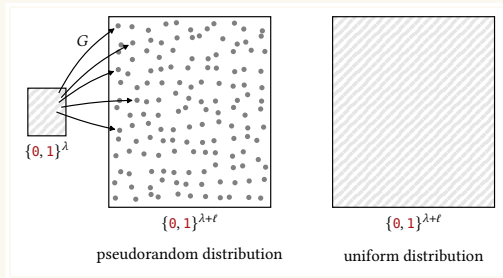
- In cryptography, something is **pseudorandom** if it is indistinguishable from a uniform distribution.^a
- We need to invent some “**secure pseudorandom generator**” (PRG) G that takes a **seed** S and ends up being indistinguishable from a true uniform distribution when thrown into a library:

^aThe Oxford English Dictionary defines the prefix pseudo- as “apparently but not really.”



Enter pseudorandomness

- In cryptography, something is **pseudorandom** if it is indistinguishable from a uniform distribution.^a
- We need to invent some “**secure pseudorandom generator**” (PRG) G that takes a **seed** S and ends up being indistinguishable from a true uniform distribution when thrown into a library:

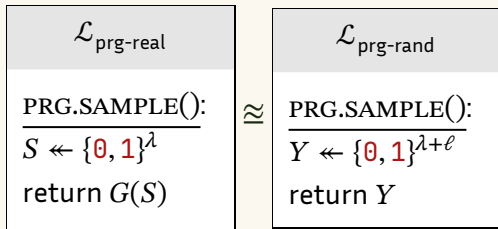


Source: The Joy of Cryptography

^aThe Oxford English Dictionary defines the prefix pseudo- as “apparently but not really.”

Enter pseudorandomness

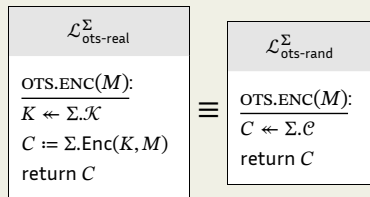
- We don't know how to make PRGs, or even if they exist.
- So we simply invent functions that we think act close enough to PRGs (when subjected to statistical and mathematical analysis).
- The concept of **computational indistinguishability** with **negligible difference** is again useful here!



One-time secrecy of a SKE

One-time Secrecy for SKE

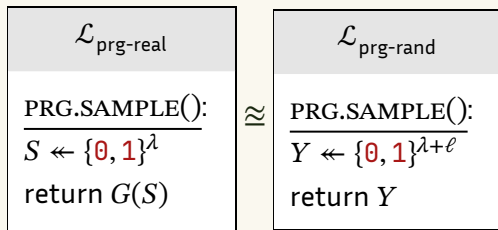
An SKE scheme Σ has one-time secrecy if the following libraries are interchangeable:



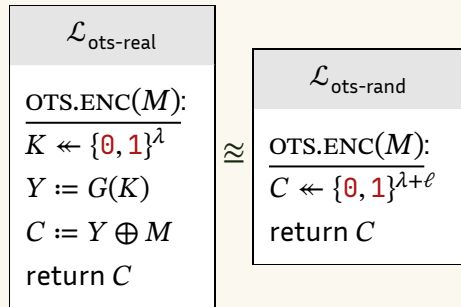
An encryption scheme has one-time secrecy if its ciphertexts are uniformly distributed, when keys are sampled uniformly, kept secret, and used for only one encryption, and no matter how the plaintexts are chosen.

Does a PRG-based encryption scheme have one-time secrecy?

If:



Then:



Attacking PRGs

- Assume that $G(S)$ is a secure PRG.
- Is $H(S)$ secure?

$H(S)$:
 $A\|B := G(S)$
 $C\|D := G(B)$
return $A\|B\|C\|D$

$\mathcal{L}_{\text{prg-real}}^H$

PRG.SAMPLE():
 $S \leftarrow \{\mathbf{0}, \mathbf{1}\}^\lambda$
 $A\|B := G(S)$
 $C\|D := G(B)$
return $A\|B\|C\|D$

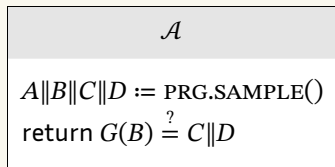
$\stackrel{?}{\approx}$

$\mathcal{L}_{\text{prg-rand}}^H$

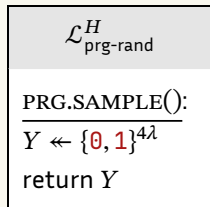
PRG.SAMPLE():
 $Y \leftarrow \{\mathbf{0}, \mathbf{1}\}^{4\lambda}$
return Y

Attacking PRGs

- Assume that $G(S)$ is a secure PRG.
- Is $H(S)$ secure?
- **No:**
- $\Pr[\mathcal{A} \diamond \mathcal{L}_{\text{prg-real}}^H \Rightarrow \text{true}] = 1$
- $\Pr[\mathcal{A} \diamond \mathcal{L}_{\text{prg-rand}}^H \Rightarrow \text{true}] = \frac{1}{2^{2\lambda}}$
- Difference certainly not negligible.



$\diamond$



Example: RC4 (a stream cipher)

- Used in WEP, SSL/TLS, and other protocols.
- Simple PRG that takes a seed and produces a keystream.
- Basic operation:
 - Initialize S-box with permutation of bytes 0-255.
 - Use key to scramble the S-box.
 - Generate pseudorandom bytes iteratively.
- Several weaknesses found over time:
 - Statistical biases in initial output.
 - Correlation between key and output bytes.
 - Considered cryptographically broken today.

Security Warning

RC4 is presented as a historical example only. It should not be used in new applications due to known weaknesses. Modern alternatives include ChaCha20.

RC4 PRG Algorithm

- RC4 generates a pseudorandom stream of bytes used to encrypt data.
- After key setup (which initializes array S), the algorithm produces keystream bytes:
- Each output byte requires simple operations:
 - Array index calculations.
 - Array value swapping.
 - Modular addition.
- Fast implementation in software.
- Despite simplicity, it has several cryptographic weaknesses.

```
1  def PRGA(S):  
2      i = 0  
3      j = 0  
4      while True:  
5          i = (i + 1) % 256  
6          j = (j + S[i]) % 256  
7          S[i], S[j] = S[j], S[i]  
8          K = S[(S[i] + S[j]) % 256]  
9          yield K  
10
```

RC4 PRG implementation in Python.

Section 2

Pseudorandom Functions

Pseudorandom function: definition

Pseudorandom Function (PRF)

A function $F : \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a secure pseudorandom function (PRF) if the following two libraries are indistinguishable:

- n is the input length of the PRF.
- m is the output length of the PRF.
- λ is the key size and hence the security parameter.

 $\mathcal{L}_{\text{prf-real}}^F$ $K \leftarrow \{0,1\}^\lambda$

PRF.QUERY(X):
return $F(K, X)$

 $\approx$ $\mathcal{L}_{\text{prf-rand}}^F$ $L := []$

PRF.QUERY(X):
if $L[X]$ undefined:
 $L[X] \leftarrow \{0,1\}^m$
return $L[X]$

PRG vs PRF: Key Differences

- **PRG (Pseudorandom Generator):**

- Takes short seed, produces longer output
- Generates entire output as a monolithic string
- $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+\ell}$

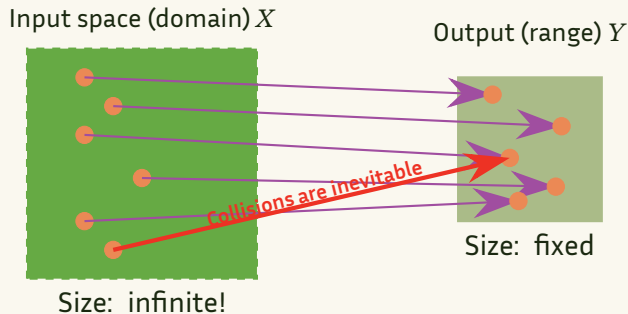
- **PRF (Pseudorandom Function):**

- Maps inputs to pseudorandom outputs
- Provides access to individual blocks of output
- Can generate output for any input on demand
- $F : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^m$
- PRFs enable “selective access” to pseudorandom values without generating the entire sequence

$$\mathcal{L}_{\text{prf-rand}}^F$$

$$\text{PRF} : F_k : X \rightarrow Y$$

- We want the mapping to be:
 - One-way
 - "Randomized"
 - Relations between inputs not reflected in outputs



PRFs in the real world: hash functions

Hash Function Properties

- Takes input of **any size**
- Produces output of **fixed size**
- Is **deterministic** (same input → same output)
- Even a **tiny change** in input creates completely different output
- Is **efficient** to compute

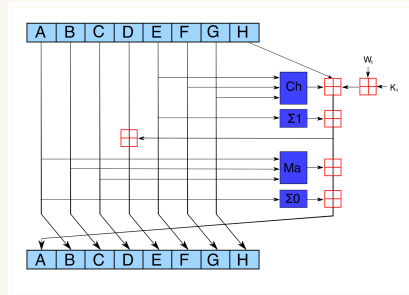
SHA256(hello) =
2cf24dba5fb0a30e26e83b2ac5
b9e29e1b161e5c1fa7425e7304
3362938b9824

SHA256(hullo) =
7835066a1457504217688c8f5d
06909c6591e0ca78c254ccf174
50d0d999cab0

Note: One character change →
completely different hash!

Expected properties of a hash function

- **Collision resistance:** computationally infeasible to find two different inputs producing the same hash.
- **Preimage resistance:** given the output of a hash function, it is computationally infeasible to reconstruct the original input.
- **Second preimage resistance:** given an input and an output, it's computationally infeasible to find another different input producing the same output.



SHA-2 compression function. Source:
Wikipedia

Hash functions: what are they good for?

- **Data integrity verification:** Hash a file. Later hash it again and compare hashes to check if the file has changed, suffered storage degradation, etc.
- **Proof of work:** Server asks client to hash something a lot of times before they can access some resource. Useful for anti-spam, Bitcoin mining, etc.

An insecure PRF construction

Claim: An Insecure PRF

The function $F(K, X) = G(K) \oplus X$ is not a secure PRF, even if G is a secure PRG.

- This construction fails because:
 - The key K is only fed through the PRG once.
 - The same value $G(K)$ is used for all queries.
 - This creates exploitable patterns in outputs.

$\mathcal{L}_{\text{prf-real}}^F$

$K \leftarrow \{0, 1\}^\lambda$

PRF.QUERY(X):
return $G(K) \oplus X$

$\not\approx$

$\mathcal{L}_{\text{prf-rand}}^F$

$L := []$

PRF.QUERY(X):
if $L[X]$ undefined:
 $L[X] \leftarrow \{0, 1\}^m$
return $L[X]$

An insecure PRF construction

- When an adversary sees two outputs:

$$Y_1 = G(K) \oplus X_1$$

$$Y_2 = G(K) \oplus X_2$$

- Taking $Y_1 \oplus Y_2$ causes $G(K)$ to cancel:

$$\begin{aligned} Y_1 \oplus Y_2 &= G(K) \oplus X_1 \oplus G(K) \oplus X_2 \\ &= X_1 \oplus X_2 \end{aligned}$$

- In a truly random function, $Y_1 \oplus Y_2 = X_1 \oplus X_2$ would be extremely unlikely!

$\mathcal{L}_{\text{prf-real}}^F$
$K \leftarrow \{0, 1\}^\lambda$
<u>PRF.QUERY(X):</u> return $G(K) \oplus X$



$\mathcal{L}_{\text{prf-rand}}^F$
$L := []$
<u>PRF.QUERY(X):</u> if $L[X]$ undefined: $L[X] \leftarrow \{0, 1\}^m$ return $L[X]$

Another insecure PRF construction

Claim: Another Insecure PRF

The function $H(K_1 \| K_2, X_1 \| X_2) = F(K_1, X_1) \oplus F(K_2, X_2)$ is not a secure PRF, even if F is a secure PRF.

- An unsuccessful attempt to use a PRF with shorter input length to build one with a larger input length.

$\mathcal{L}_{\text{prf-real}}^H$

$K_1 \| K_2 \leftarrow \{0, 1\}^{2\lambda}$

PRF.QUERY($X_1 \| X_2$):

return $F(K_1, X_1) \oplus F(K_2, X_2)$

$\approx$

$\mathcal{L}_{\text{prf-rand}}^H$

$L := []$

PRF.QUERY($X_1 \| X_2$):

if $L[X_1 \| X_2]$ undefined:

$L[X_1 \| X_2] \leftarrow \{0, 1\}^m$

return $L[X_1 \| X_2]$

Why the previous PRF is broken

- When inputs share the same first half, the corresponding outputs of H have a common term $F(K_1, X_1)$
- Consider querying four inputs: $A\|B$, $A\|B'$, $A'\|B$, $A'\|B'$ (where $A \neq A'$ and $B \neq B'$)

$$Y_1 = F(K_1, A) \oplus F(K_2, B)$$

$$Y_2 = F(K_1, A) \oplus F(K_2, B')$$

$$Y_3 = F(K_1, A') \oplus F(K_2, B)$$

$$Y_4 = F(K_1, A') \oplus F(K_2, B')$$

- If we XOR $Y_1 \oplus Y_2$, the $F(K_1, A)$ terms cancel out.
- Similarly, $Y_3 \oplus Y_4$ causes $F(K_1, A')$ to cancel:
 - $Y_1 \oplus Y_2 = F(K_2, B) \oplus F(K_2, B')$
 - $Y_3 \oplus Y_4 = F(K_2, B) \oplus F(K_2, B')$
- So $\Pr[Y_1 \oplus Y_2 \stackrel{?}{=} Y_3 \oplus Y_4] = 1$
- With a truly random function, $\Pr[Y_1 \oplus Y_2 \stackrel{?}{=} Y_3 \oplus Y_4] = \frac{1}{2^m}$ (extremely unlikely!)
- Also, given $Y_1 \dots Y_3$, we can predict Y_4 !

The Golden Rule of PRFs

The Golden Rule of PRFs

If a PRF F is being used as a component in a larger construction H , then security usually rests on how well H can ensure distinct inputs to F .

- When analyzing PRF security, focus on input uniqueness.
- Repeated inputs to a PRF create exploitable patterns.
- Even if F is secure, H can be broken if it causes F to receive duplicate inputs.
- Don't try to directly distinguish F 's outputs from uniform.
- Instead, exploit how H uses F incorrectly.
- Find input patterns that force collisions within F .

Section 3

Pseudorandom Permutations

What is a permutation?

Permutation

A permutation is a rearrangement where each input value maps to exactly one output value, and each possible output appears exactly once.

- Permutations rearrange elements rather than transforming them.
- Every element in the domain appears exactly once in the range.
- The function is invertible.

- **Example:** simple substitution cipher. Each letter maps to another letter:

$$\begin{aligned} a &\mapsto g, & b &\mapsto a, & c &\mapsto r \\ d &\mapsto b, & e &\mapsto l, & \dots \end{aligned}$$

- Under this permutation:
 - "cabbage" $\mapsto$ "rgaagdl"
 - "rgaagdl" $\mapsto$ "cabbage"
- This mapping is reversible because it's a permutation over $\{a, \dots, z\}$.
- Each letter appears exactly once in the output alphabet.

PRF versus PRP

Pseudo-Random Function (HMAC-SHA-2)

- **Input** is arbitrary-length,
- **Output** is fixed-length, looks random (as discussed earlier).
- Indistinguishable from a truly random function by an adversary with limited computational power.

Pseudo-Random Permutation (AES)

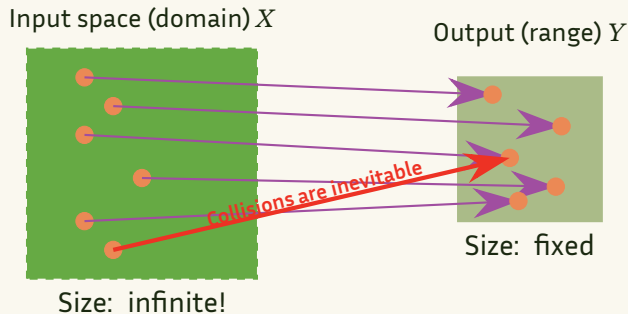
- **Input and output** are the same length, forming a permutation.
- Each input maps uniquely to one output, allowing invertibility.
- Indistinguishable from a truly random permutation by an adversary with limited computational power.

PRPs compared to PRFs

- **Invertibility:** PRPs can be efficiently inverted given the key
 - Enable both encryption and decryption
 - Can recover input from output (and vice versa)
- **No range collision:** Each input maps to a unique output
 - Provides perfect input recovery
 - Reduces vulnerability to collision-based attacks, birthday attacks, and certain forms of differential cryptanalysis
- **Versatility:** A secure PRP can be used as a PRF
 - “Downgrade” trivially by ignoring inverse capability
 - The reverse is not true (PRF $\rightarrow$ PRP conversion is complex)

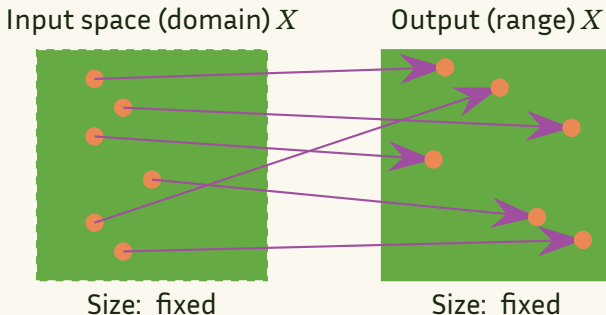
$$\text{PRF} : F_k : X \rightarrow Y$$

- We want the mapping to be:
 - One-way
 - "Randomized"
 - Relations between inputs not reflected in outputs



$$\text{PRP} : F_k : X \rightarrow X$$

- **Bijective** (two-way)
 - **Injective**: no two inputs map to same output (no collisions)
 - **Surjective**: Every output has one corresponding input
- “Randomized”
- Relations between inputs not reflected in outputs



“Lazy dictionaries” versus “lazy permutations”

Ideal PRF versus ideal PRP

$\mathcal{L}_{\text{prf-rand}}^F$

$L := []$

PRF.QUERY(X):

if $L[X]$ undefined:

$L[X] \leftarrow \{0, 1\}^m$

return $L[X]$

$\mathcal{L}_{\text{prp-rand}}^F$

$L := []$

PRP.QUERY(X):

if $L[X]$ undefined:

$Y \leftarrow \{0, 1\}^n \setminus y$

$y := y \cup \{Y\}$

$L[X] := Y$

return $L[X]$

“Lazy dictionaries” versus “lazy permutations”

- While the PRF (on the left) just picks random outputs for each input...
- The PRP (on the right) must ensure outputs are never repeated:
 - y tracks all outputs used so far
 - $\setminus$ means “set difference” - pick from values not in y
 - $\cup$ means “set union” - add the new value to y
- This ensures each output appears exactly once - the definition of a permutation

$\mathcal{L}_{\text{prp-rand}}^F$

$L := []$

PRP.QUERY(X):

if $L[X]$ undefined:

$Y \leftarrow \{0, 1\}^n \setminus y$

$y := y \cup \{Y\}$

$L[X] := Y$

return $L[X]$

Pseudorandom function: definition

Pseudorandom Function (PRF)

A function $F : \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a secure pseudorandom function (PRF) if the following two libraries are indistinguishable:

- n is the input length of the PRF.
- m is the output length of the PRF.
- λ is the key size and hence the security parameter.

$\mathcal{L}_{\text{prf-real}}^F$

$K \leftarrow \{0,1\}^\lambda$

PRF.QUERY(X):
return $F(K, X)$

$\approx$

$\mathcal{L}_{\text{prf-rand}}^F$

$L := []$

PRF.QUERY(X):
if $L[X]$ undefined:
 $L[X] \leftarrow \{0,1\}^m$
return $L[X]$

Pseudorandom permutation: definition

Pseudorandom Permutation (PRP)

A keyed permutation $F^\pm$ is a secure pseudorandom permutation (PRP) if the following two libraries are indistinguishable:

- n is the input length of the PRP. Also the output length!
- λ is the key size and hence the security parameter.

$\mathcal{L}_{\text{prp-real}}^F$

$K \leftarrow \{0, 1\}^\lambda$

PRP.QUERY(X):
return $F(K, X)$

$\approx$

$\mathcal{L}_{\text{prp-rand}}^F$

$L := []$

PRP.QUERY(X):

if $L[X]$ undefined:

$Y \leftarrow \{0, 1\}^n \setminus y$

$y := y \cup \{Y\}$

$L[X] := Y$

return $L[X]$

Pseudorandom permutation: definition

Pseudorandom Permutation (PRP)

A keyed permutation $F^\pm$ is a secure pseudorandom permutation (PRP) if the following two libraries are indistinguishable:

- We obviously can't use $\mathcal{L}_{\text{prp-rand}}^F$ in the real world.
- It doesn't scale. So we need practical approximative alternatives.

$\mathcal{L}_{\text{prp-real}}^F$

$K \leftarrow \{0, 1\}^\lambda$

PRP.QUERY(X):
return $F(K, X)$

$\approx$

$\mathcal{L}_{\text{prp-rand}}^F$

$L := []$

PRP.QUERY(X):

if $L[X]$ undefined:

$Y \leftarrow \{0, 1\}^n \setminus y$

$y := y \cup \{Y\}$

$L[X] := Y$

return $L[X]$

Building a permutation through Feistel ciphers

- An r -round **Feistel cipher** with **round functions** $F_1, \dots, F_r$ is defined as follows:

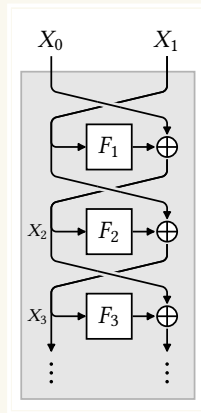
$\mathbb{F}(X_0 \| X_1)$:

for $i = 1$ to r :

$$X_{i+1} := X_{i-1} \oplus F_i(X_i)$$

return $X_r \| X_{r+1}$

- A Feistel cipher is always a permutation on $\{0, 1\}^{2n}$, regardless of its round functions.



Feistel cipher.

Source: The Joy of Cryptography

Building a permutation through Feistel ciphers

Feistel Ciphers Are Permutations

A Feistel cipher is always a permutation on $\{0, 1\}^{2n}$, regardless of its round functions.

- **Proof:** Each round of a Feistel cipher computes the next block as:
 - $X_{i+1} = X_{i-1} \oplus F_i(X_i)$
- To invert this round, we can rearrange the equation to solve for X_{i-1} in terms of X_i and X_{i+1} :
 - $X_{i-1} = X_{i+1} \oplus F_i(X_i)$
- F_i itself does not need to have an inverse - both the forward and inverse direction of the Feistel cipher evaluate F_i in the forward direction! $\square$

Building a permutation through Feistel ciphers

Inversion

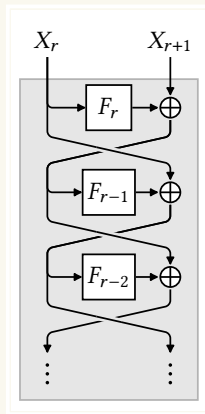
- $\mathbb{F}^{-1}$ is the **inversion**, allowing Feistel ciphers to function as PRPs:

$\mathbb{F}^{-1}(X_r \| X_{r+1})$:

for $i = r$ **down to** 1:

$X_{i-1} := X_{i+1} \oplus F_i(X_i)$

return $X_0 \| X_1$



Feistel cipher inversion.
Source: The Joy of Cryptography

Forward and backward on one slide

Just in case it's helpful

$\mathbb{F}(X_0 \| X_1)$:

for $i = 1$ to r :

$X_{i+1} := X_{i-1} \oplus F_i(X_i)$

return $X_r \| X_{r+1}$

$\mathbb{F}^{-1}(X_r \| X_{r+1})$:

for $i = r$ down to 1:

$X_{i-1} := X_{i+1} \oplus F_i(X_i)$

return $X_0 \| X_1$

Keyed Feistel ciphers

- Let's add a key in there. Wow, encryption!
- $K_{1\dots i}$ is called the **key schedule**.
- If each round function F_i uses a distinct key K_i , it increases the security of the Feistel network against certain attacks.
- By using a PRF as round function F_i , the security of the Feistel cipher would be grounded on the PRF's security basis.

$\mathbb{F}(K_1 \parallel \dots \parallel K_r, X_0 \parallel X_1)$:

for $i = 1$ to r :

$X_{i+1} := X_{i-1} \oplus F_i(K_i, X_i)$

return $X_r \parallel X_{r+1}$

Meet-in-the-middle attacks on Feistel ciphers

Why use a key schedule and not the same key?

- Using the **same key** for all rounds creates significant vulnerabilities.
- A meet-in-the-middle attack can break an r -round Feistel cipher with complexity $< 2^{|K|}$.
- The attack works by computing partial encryptions from both ends:
 1. Forward: Compute halfway through encryption.
 2. Backward: Compute halfway through decryption.
 3. Look for “meeting points” in the middle.
- With identical round functions, effective security may be only **half** the number of rounds.

$\mathbb{F}(K, X_0 \| X_1)$:

for $i = 1$ to r :

$$X_{i+1} := X_{i-1} \oplus F(K, X_i)$$

return $X_r \| X_{r+1}$

Breaking a 2-round Feistel cipher

- A 2-round Feistel cipher cannot be a secure PRP.

$$\begin{array}{l} \mathbb{F}(K_1 \| K_2, X_0 \| X_1): \\ \hline X_2 := X_0 \oplus F(K_1, X_1) \\ X_3 := X_1 \oplus F(K_2, X_2) \\ \text{return } X_2 \| X_3 \end{array}$$

Breaking a 2-round Feistel cipher

- Once again, real or random?
- We can indeed produce an adversary $\mathcal{A}$ that can distinguish between these two libraries.

$\mathcal{L}_{\text{prp-real}}^{\mathbb{F}}$

$K_1 \| K_2 \leftarrow \{0, 1\}^{2\lambda}$

$\text{PRP.QUERY}_{\mathbb{F}}(X_0 \| X_1):$

$X_2 := X_0 \oplus F(K_1, X_1)$

$X_3 := X_1 \oplus F(K_2, X_2)$

return $X_2 \| X_3$

$\not\approx$

$\mathcal{L}_{\text{prp-rand}}^{\mathbb{F}}$

$L := []$

$\text{PRP.QUERY}_{\mathbb{F}}(X):$

if $L[X]$ undefined:

$Y \leftarrow \{0, 1\}^n \setminus y$

$y := y \cup \{Y\}$

$L[X] := Y$

return $L[X]$

Breaking a 2-round Feistel cipher

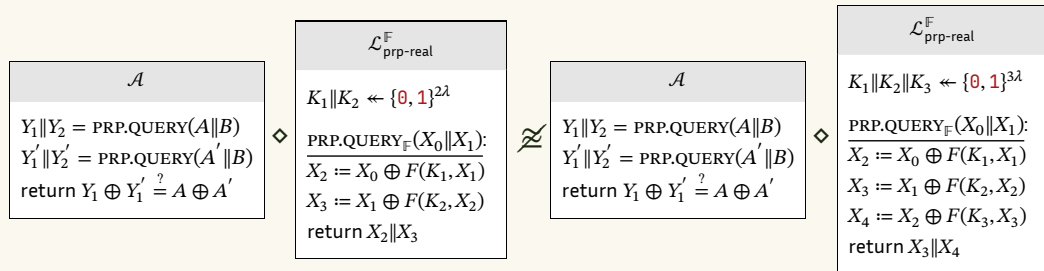
- Let's query $(A\|B)$ and $(A'\|B)$ where $A \neq A'$
 - $Y_1\|Y_2 = \text{PRP.QUERY}(A\|B)$
 - $Y'_1\|Y'_2 = \text{PRP.QUERY}(A'\|B)$
- In a 2-round Feistel cipher:
 - $Y_1 = A \oplus F(K_1, B)$
 - $Y'_1 = A' \oplus F(K_1, B)$
 - $Y_1 \oplus Y'_1 = A \oplus A'$
- In a true PRP,
 $\Pr[Y_1 \oplus Y'_1 \stackrel{?}{=} A \oplus A']$ is negligible

$\mathcal{A}$
$Y_1\ Y_2 = \text{PRP.QUERY}(A\ B)$ $Y'_1\ Y'_2 = \text{PRP.QUERY}(A'\ B)$ return $Y_1 \oplus Y'_1 \stackrel{?}{=} A \oplus A'$



$\mathcal{L}_{\text{prp-real}}^{\mathbb{F}}$
$K_1\ K_2 \leftarrow \{0, 1\}^{2\lambda}$ $\text{PRP.QUERY}_{\mathbb{F}}(X_0\ X_1):$ $X_2 := X_0 \oplus F(K_1, X_1)$ $X_3 := X_1 \oplus F(K_2, X_2)$ return $X_2\ X_3$

However, a 3+ round Feistel cipher is fine!

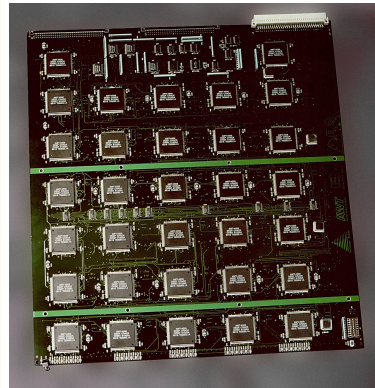


DES: Feistel in practice

- **Data Encryption Standard (DES)** is a classic real-world implementation of the Feistel structure
- Properties:
 - 16 Feistel rounds.
 - 56-bit key (64 bits with parity).
 - 64-bit block size.
 - Standardized in 1977.
- Problem: By the 1990s, 56-bit keys became too small for security.
- **Triple DES (3DES)** replaced it:
 - Uses three DES operations in sequence.
 - $C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$
 - Effectively doubles the key length.
 - Compatible with legacy DES when $K_1 = K_2 = K_3$
- 3DES still used in legacy systems, but largely replaced by AES for performance reasons.
- Lesson: Feistel structure made it possible to adapt DES rather than abandon it.

EFF's "Deep Crack"

- In 1998, the Electronic Frontier Foundation built a special-purpose machine called "Deep Crack".
- Cost: Only \$250,000 (far less than the NSA budget!)
- Purpose: Prove that 56-bit DES keys were insufficient.



EFF's "Deep Crack"

EFF's "Deep Crack"

- July 1998: Deep Crack broke a DES challenge in just 56 hours.
- Message revealed: "It's time for those 128-, 192-, and 256-bit keys."
- Impact:
 - Publicly demonstrated DES was obsolete.
 - Accelerated adoption of AES.
 - Made "it's theoretically breakable" a practical reality.
 - Priceless reaction from government officials!



Paul Kocher. He's still active in cryptography today, insanely productive research career, many crazy attacks

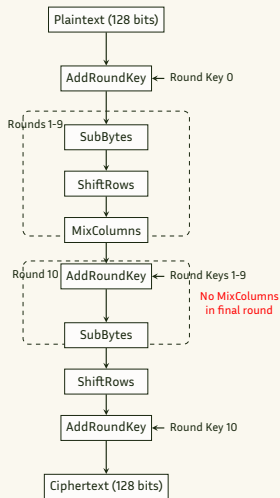
AES: a good example of a PRP

- AES is the most widely used PRP in the world.
- It works on fixed-size blocks: 128 bits.
- Key sizes: 128, 192, or 256 bits.
- Each AES key defines a specific permutation over the space of all 128-bit values.
- For each key, AES maps each possible 128-bit input to exactly one 128-bit output.
- Different keys create different permutations.
- AES is efficiently invertible:
 - $\text{Dec}(K, \text{Enc}(K, M)) = M$
- AES is believed to be computationally indistinguishable from a random permutation.
- Has withstood extensive cryptanalysis for over 20 years.

AES structure

- Internal structure: substitution-permutation network with multiple rounds.^a
 - SubBytes: non-linear substitution
 - ShiftRows: transposition
 - MixColumns: mixing operation
 - AddRoundKey: XOR with round key

^aCheck out this amazing interactive animation of AES's internal structure:
https://formaestudio.com/rijndaelinspector/archivos/Rijndael_Animation_v4_eng-html5.html



AES: security and attacks over time

- AES has been heavily analyzed for over 20 years.
- Best attacks against full AES have gradually improved:
 - 2011: Biclique attack (Bogdanov et al.) reduced complexity to $2^{126.1}$ for AES-128.
 - Various side-channel attacks developed (power analysis, cache timing).^a
 - Advances in meet-in-the-middle and related-key techniques.
- Despite these advances:
 - No practical attacks on full AES-128.
 - Best attacks still require $\approx 2^{126}$ operations.
 - At this complexity, attacks remain purely theoretical.
 - Would require resources far exceeding global computing power.
- Even quantum computers offer only modest advantage (Grover's algorithm reduces security to 2^{64} operations).^a

^aThis is the main way to attack AES in practice. Side-channel attacks will be discussed in more depth later in the course.

^aMore on quantum computers and how they affect cryptography later in the course.



Applied Cryptography

Summer 2026

Part 1: Provable Security

1.4: Pseudorandomness

Nadim Kobeissi

<https://appliedcryptography.page>